



Web-Development (Frontend)

Name : PRANAV

Roll. No : CSJMA22001390016

Branch : B-tech CSE 4th year

Objective- Simple Blogger Website using React

1

Industry Norms

ReactJS is the backbone of user interfaces for tech giants like Facebook, Instagram, and Airbnb, proving its scalability and reliability. Its widespread adoption underscores its robustness for both small projects and large-scale applications, making it a trusted choice for millions of developers globally.

2

SEO Ranking

React promotes building UIs with independent, reusable components. This modular approach significantly speeds up development, enhances code maintainability, and fosters consistency across different parts of an application, allowing for rapid iteration and scalable design.

3

Performance Optimization E6+ Norms

React's innovative Virtual DOM (Document Object Model) dramatically boosts application performance. Instead of directly manipulating the browser's DOM, React first updates a virtual representation, then efficiently calculates the minimum changes needed, leading to faster rendering and a smoother user experience.

4

Reusability

Beyond the library itself, React boasts an extensive ecosystem of tools, libraries (like Redux for state management and React Router for navigation), and a massive, active community. This robust support system means developers have access to abundant resources, solutions, and continuous innovation.



Unlocking Frontend Power with ReactJS: Benefits, Components & Props

Welcome to this deep dive into ReactJS 19v, the revolutionary JavaScript library that has transformed frontend web development. In this presentation, we'll explore why ReactJS is the go-to choice for building dynamic and efficient user interfaces, delving into its core concepts like components and props. Get ready to understand how React empowers developers to create seamless and high-performing web applications.

Why ReactJS? The Frontend Game-Changer

1

Industry Adoption & Trust

ReactJS is the backbone of user interfaces for tech giants like Facebook, Instagram, and Airbnb, proving its scalability and reliability. Its widespread adoption underscores its robustness for both small projects and large-scale applications, making it a trusted choice for millions of developers globally.

2

Component-Based Architecture

React promotes building UIs with independent, reusable components. This modular approach significantly speeds up development, enhances code maintainability, and fosters consistency across different parts of an application, allowing for rapid iteration and scalable design.

3

Performance Optimization

React's innovative Virtual DOM (Document Object Model) dramatically boosts application performance. Instead of directly manipulating the browser's DOM, React first updates a virtual representation, then efficiently calculates the minimum changes needed, leading to faster rendering and a smoother user experience.

4

Vibrant Ecosystem & Community

Beyond the library itself, React boasts an extensive ecosystem of tools, libraries (like Redux for state management and React Router for navigation), and a massive, active community. This robust support system means developers have access to abundant resources, solutions, and continuous innovation.

React Components: Building Blocks of UI

At the heart of every React application are components. Think of them as independent, self-contained modules that encapsulate both the structure (HTML-like JSX) and the logic (JavaScript) needed to render a piece of the user interface. Components allow developers to break down complex UIs into smaller, manageable, and reusable parts.

- Components are essentially JavaScript functions or classes that return React elements, which describe what should appear on the screen.
- While traditionally both function and class components were used, modern React development heavily favors function components due to hooks, offering a simpler and more concise way to manage state and side effects.
- Each component manages its own internal state and lifecycle, making it a powerful tool for building dynamic and interactive UIs.
- The true power of components lies in their composability: you can nest components within other components to build intricate UI hierarchies, much like assembling LEGO bricks to create a complex structure.



Props: The Communication Channel Between Components



Data Flow

Props, short for properties, are how data is passed from a parent component down to its child components. They serve as the primary mechanism for components to communicate and share information within the React component tree.



Read-Only Nature

A fundamental principle of props is that they are read-only. A child component cannot directly modify the props it receives from its parent. This "unidirectional data flow" ensures predictability and makes debugging much easier by preventing unexpected side effects.



Analogy

You can think of props as function arguments. Just as you pass arguments to a JavaScript function to influence its behavior, you pass props to a React component to customize its appearance or behavior.



Practical Example

Consider a `Welcome` component. To personalize it, you pass a `name` prop: `<Welcome name="Sara" />`. Inside the `Welcome` component, you can then access `props.name` to display "Hello, Sara!".

Passing Props: Simple Yet Powerful

Props are incredibly versatile, allowing you to pass almost any JavaScript data type from parent to child components. This flexibility is key to building dynamic and configurable UIs. Here are some common examples of what you can pass as props:

- **Strings:** For text content, like names or messages. `<Button text="Click Me" />`
- **Numbers:** For numerical values, like counts or IDs. `<Product quantity={5} />`
- **Booleans:** For true/false flags, often used to control visibility or state. `<MenuItem isActive={true} />`
- **Objects:** For structured data, like user profiles or product details. `<UserCard user={{ name: "John Doe", age: 30 }} />`
- **Arrays:** For lists of items. `<TodoList items={['Buy milk', 'Walk dog']} />`
- **Functions:** To allow child components to trigger actions in the parent (e.g., event handlers). `<Button onClick={handleClick} />`
- **React Elements (Children Prop):** Content nested between component tags is passed as a special `children` prop. `<Card><h2>Title</h2><p>Content</p></Card>`

This example demonstrates passing a string, a number, and a function as props to a custom `MyButton` component:

```
function MyButton(props) { return ( <button onClick={props.onPress}> {props.label} ({props.count}) </button> );}<MyButton label="Clicks" count={0} onPress={() => console.log("Button clicked!")} />
```

Key Takeaways & Next Steps

React's Power

ReactJS streamlines frontend development with its component-based architecture and efficient Virtual DOM, enabling fast and interactive user interfaces.

Props

Props facilitate one-way data flow from parent to child components, ensuring predictable and maintainable communication within your application.



Components

Components are reusable, self-contained UI building blocks that manage their own logic and structure, making complex UIs manageable.



Dive Deeper

To truly master React, explore state management (with `useState/useReducer`), lifecycle methods (with `useEffect`), and routing with React Router.

ReactJS offers a robust and flexible foundation for modern web development. By understanding these core concepts, you are well-equipped to build powerful and engaging frontend applications. Continue experimenting, building, and exploring the vast React ecosystem!

[Explore React Documentati](#)[Visit React on GitHub](#)

- public
 - ★ favicon.ico
 - <> index.html
 - 🖼️ logo192.png
 - 🖼️ logo512.png
 - { } manifest.json
 - ☰ robots.txt
- src
 - # App.css
 - JS App.js
 - JS App.test.js
 - JS Blogs.js
 - JS Footer.js
 - JS Home.js
 - # index.css
 - JS index.js
 - 🖼️ logo.svg
 - JS Navbar.js
 - JS Post.js
 - JS reportWebVitals.js
 - JS setupTests.js
 - # styles.css
 - { } package-lock.json
 - { } package.json

```

1  import './App.css';
2  import Navbar from './Navbar';
3  import Home from './Home';
4  import Post from './Post';
5  import Footer from './Footer';
6  import { Route, Routes } from 'react-router-dom';
7
8  function App() {
9    return (
10     <div className="App">
11       <Navbar></Navbar>
12       <Routes>
13         <Route path="/" element={<Home />} />
14         <Route path="/post" element={<Post />} />
15       </Routes>
16       <Footer />
17     </div>
18   );
19 }
20
21 export default App;

```

```

1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6  import { BrowserRouter } from "react-router-dom";
7
8  const root = ReactDOM.createRoot(document.getElementById("root"));
9  root.render(
10   <BrowserRouter>
11     <App />
12   </BrowserRouter>
13 );
14

```

```

1  import React, { useState, useEffect } from "react";
2  import Blogs from './Blogs';
3  import { useNavigate, useLocation } from "react-router-dom";
4
5  export default function Home() {
6    const [blogs, setBlogs] = useState([
7      { _id: "1", title: "No Blog is Available", content: "....." },
8    ]);
9    const navigate = useNavigate();
10   const location = useLocation();
11
12   function handleClick() {
13     navigate("/post");
14   }
15
16   useEffect(() => {
17     fetch("/blogs")
18       .then((response) => response.json())
19       .then((data) => {
20         if (data.length !== 0) {
21           setBlogs(data);
22         }
23       })
24       .catch((error) => {
25         console.error("Error fetching blogs:", error);
26       });
27   }, [location.state?.refresh]);
28
29   return (
30     <>
31       {blogs.map((blog) => (
32         <Blogs
33           key={blog._id}
34           title={blog.title}
35           content={blog.content}
36           id={blog._id}
37         />

```

No Blog is Available

.....

Edit 

Delete

Comments:

Failed to fetch comments

No Comment Available

Add Comment

Add New Post 

This is My First Blog

This is Going to my First Blog ^_^

[Edit](#) [Delete](#)

Comments:

Nice Post Bro

[Add Comment](#)[Add New Post](#) 

Thank You